

一、问题背景

iptables 用户空间工具 (v1.8.7) 编译链接正确，但部署到 Hi3519DV500 设备后，大部分扩展匹配模块无法使用：

```
# iptables -m conntrack --ctstate ESTABLISHED -j ACCEPT
iptables v1.8.7 (legacy): Couldn't find match `conntrack'

# iptables -m state --state ESTABLISHED -j ACCEPT
iptables v1.8.7 (legacy): Couldn't find match `state'

# cat /proc/net/ip_tables_matches
mark
icmp
udplite
udp
tcp
```

只有 5 个基本匹配可用 (mark、icmp、udplite、udp、tcp)，其余 60+ 个扩展全部“找不到”。

问题定位： 用户空间 iptables 编译和链接完全正确 (nm 符号、反汇编均确认 init_extensions() 调用链无误)，根因在于内核未编译对应的 netfilter 扩展模块。

二、根因分析

2.1 iptables 扩展加载机制

iptables 用户空间采用静态全量链接模式 (NO_SHARED_LIBS=1 + ALL_INCLUSIVE)，所有 60+ 个扩展的 libxt_xxx_init() 函数编译到同一个二进制文件中。启动时通过 init_extensions() → xtables_register_match() 注册到内存链表。

但注册过程有一个关键校验步骤：

```
xtables_register_match()
└─ 加入 xtables_pending_matches 链表 (pending)

xtables_find_match("conntrack")
└─ 遍历 pending 链表，找到 conntrack
└─ 调用 xtables_fully_register_pending_match()
    └─ 调用 compatible_match_revision("conntrack", revision)
        └─ socket(AF_INET, SOCK_RAW, IPPROTO_RAW)
        └─ getsockopt(sockfd, IPPROTO_IP, IPT_SO_GET_REVISION_MATCH, &rev)
            └─ 内核返回 ENOENT (模块不存在)
            └─ return 0 (不支持)
        └─ return false (注册失败)
└─ conntrack 留在 pending 链表，不会进入活跃列表
└─ 最终报告 "Couldn't find match `conntrack'"
```

核心机制： compatible_match_revision() 通过 getsockopt() 系统调用向内核查询该扩展是否被支持。内核只有编译了对应的 CONFIG_NETFILTER_XT_MATCH_xxx 选项后才会返回成功。

2.2 原始内核配置状态

文件: `kernel/linux/config/linux-5.10/arch/arm64/configs/hispanic_hi3519dv500_small_defconfig`

原始 defconfig 中, 几乎所有 `CONFIG_NETFILTER_XT_MATCH_*` 选项都被禁用:

```
# CONFIG_NETFILTER_XT_MATCH_CONNTRACK is not set    ← conntrack 无法使用
# CONFIG_NETFILTER_XT_MATCH_STATE is not set         ← state 无法使用
# CONFIG_NETFILTER_XT_MATCH_COMMENT is not set       ← comment 无法使用
# CONFIG_NETFILTER_XT_MATCH_LIMIT is not set         ← limit 无法使用
# CONFIG_NETFILTER_XT_MATCH_MAC is not set           ← mac 无法使用
# CONFIG_NETFILTER_XT_MATCH_MULTIPORT is not set     ← multiport 无法使用
... (其余全部 is not set)
```

仅有以下选项启用:

- `CONFIG_NETFILTER_XT_MARK=y` — mark target/match
- 内置的 TCP/UDP/ICMP 匹配 (由 `ip_tables` 模块自带)

三、修复步骤

3.1 修改内核 defconfig

文件: `kernel/linux/config/linux-5.10/arch/arm64/configs/hispanic_hi3519dv500_small_defconfig`

3.1.1 启用 conntrack 基础依赖

conntrack 是连接跟踪的核心功能, 许多其他模块依赖它。原配置中 `CONFIG_NF_CONNTRACK=y` 已启用, 但 `NF_CONNTRACK_MARK` 未启用:

```
- # CONFIG_NF_CONNTRACK_MARK is not set
+ CONFIG_NF_CONNTRACK_MARK=y
```

3.1.2 启用 xt connmark 框架

connmark 是 conntrack 和 mark 的桥梁, target 和 match 都需要:

```
- # CONFIG_NETFILTER_XT_CONNMARK is not set
+ CONFIG_NETFILTER_XT_CONNMARK=y
```

3.1.3 启用常用 target 模块

```
- # CONFIG_NETFILTER_XT_TARGET_CLASSIFY is not set
+ CONFIG_NETFILTER_XT_TARGET_CLASSIFY=y
- # CONFIG_NETFILTER_XT_TARGET_CONNMARK is not set
+ CONFIG_NETFILTER_XT_TARGET_CONNMARK=y
- # CONFIG_NETFILTER_XT_TARGET_LOG is not set
+ CONFIG_NETFILTER_XT_TARGET_LOG=y
- # CONFIG_NETFILTER_XT_TARGET_MARK is not set
+ CONFIG_NETFILTER_XT_TARGET_MARK=y
- # CONFIG_NETFILTER_XT_TARGET_NFLOG is not set
+ CONFIG_NETFILTER_XT_TARGET_NFLOG=y
- # CONFIG_NETFILTER_XT_TARGET_NFQUEUE is not set
+ CONFIG_NETFILTER_XT_TARGET_NFQUEUE=y
```

3.1.4 启用常用 match 模块

```
- # CONFIG_NETFILTER_XT_MATCH_ADDRTYPE is not set
+ CONFIG_NETFILTER_XT_MATCH_ADDRTYPE=y
- # CONFIG_NETFILTER_XT_MATCH_COMMENT is not set
+ CONFIG_NETFILTER_XT_MATCH_COMMENT=y
- # CONFIG_NETFILTER_XT_MATCH_CONNMARK is not set
+ CONFIG_NETFILTER_XT_MATCH_CONNMARK=y
- # CONFIG_NETFILTER_XT_MATCH_CONNTRACK is not set
+ CONFIG_NETFILTER_XT_MATCH_CONNTRACK=y
- # CONFIG_NETFILTER_XT_MATCH_DSCP is not set
+ CONFIG_NETFILTER_XT_MATCH_DSCP=y
- # CONFIG_NETFILTER_XT_MATCH_ESP is not set
+ CONFIG_NETFILTER_XT_MATCH_ESP=y
- # CONFIG_NETFILTER_XT_MATCH_HELPER is not set
+ CONFIG_NETFILTER_XT_MATCH_HELPER=y
- # CONFIG_NETFILTER_XT_MATCH_IPRANGE is not set
+ CONFIG_NETFILTER_XT_MATCH_IPRANGE=y
- # CONFIG_NETFILTER_XT_MATCH_LENGTH is not set
+ CONFIG_NETFILTER_XT_MATCH_LENGTH=y
- # CONFIG_NETFILTER_XT_MATCH_LIMIT is not set
+ CONFIG_NETFILTER_XT_MATCH_LIMIT=y
- # CONFIG_NETFILTER_XT_MATCH_MAC is not set
+ CONFIG_NETFILTER_XT_MATCH_MAC=y
- # CONFIG_NETFILTER_XT_MATCH_MARK is not set
+ CONFIG_NETFILTER_XT_MATCH_MARK=y
- # CONFIG_NETFILTER_XT_MATCH_MULTIPORT is not set
+ CONFIG_NETFILTER_XT_MATCH_MULTIPORT=y
- # CONFIG_NETFILTER_XT_MATCH_OWNER is not set
+ CONFIG_NETFILTER_XT_MATCH_OWNER=y
- # CONFIG_NETFILTER_XT_MATCH_PKTTYPE is not set
+ CONFIG_NETFILTER_XT_MATCH_PKTTYPE=y
- # CONFIG_NETFILTER_XT_MATCH_QUOTA is not set
+ CONFIG_NETFILTER_XT_MATCH_QUOTA=y
- # CONFIG_NETFILTER_XT_MATCH_STATE is not set
+ CONFIG_NETFILTER_XT_MATCH_STATE=y
- # CONFIG_NETFILTER_XT_MATCH_STATISTIC is not set
```

```
+ CONFIG_NETFILTER_XT_MATCH_STATISTIC=y
- # CONFIG_NETFILTER_XT_MATCH_STRING is not set
+ CONFIG_NETFILTER_XT_MATCH_STRING=y
- # CONFIG_NETFILTER_XT_MATCH_TCPMSS is not set
+ CONFIG_NETFILTER_XT_MATCH_TCPMSS=y
- # CONFIG_NETFILTER_XT_MATCH_TIME is not set
+ CONFIG_NETFILTER_XT_MATCH_TIME=y
- # CONFIG_NETFILTER_XT_MATCH_U32 is not set
+ CONFIG_NETFILTER_XT_MATCH_U32=y
```

3.2 已有配置（无需修改）

以下选项在原始 defconfig 中已经启用，无需额外修改：

```
CONFIG_NETFILTER=y
CONFIG_NETFILTER_ADVANCED=y
CONFIG_NF_CONNTRACK=y
CONFIG_NF_NAT=y
CONFIG_NF_NAT_REDIRECT=y
CONFIG_NF_NAT_MASQUERADE=y
CONFIG_NETFILTER_XTABLES=y
CONFIG_NETFILTER_XT_MARK=y
CONFIG_NETFILTER_XT_NAT=y
CONFIG_NETFILTER_XT_TARGET_NETMAP=y
CONFIG_NETFILTER_XT_TARGET_REDIRECT=y
CONFIG_NETFILTER_XT_TARGET_MASQUERADE=y
CONFIG_IP_NF_FILTER=y
CONFIG_IP_NF_TARGET_REJECT=y
CONFIG_IP_NF_NAT=y
CONFIG_IP_NF_MANGLE=y
```

3.3 重新编译

```
# 全量编译（内核配置变更需要重新编译内核）
rm -rf out && hb build -f
```

编译完成后，将生成的内核镜像烧录到设备。

四、修改汇总

修改文件清单

文件	修改内容
kernel/linux/config/linux-5.10/arch/arm64/configs/hispanic_hi3519dv500_small_defconfig	启用 25 个 NETFILTER 扩展选项（1 个 conntrack 依赖 + 1 个 xt core + 6 个 target + 17 个 match）

新增启用的内核选项一览

类别	选项	支持的 iptables 功能
conntrack 依赖	NF_CONNTRACK_MARK	连接跟踪与 mark 联动
xt core	XT_CONNMARK	connmark target/match 框架
target	XT_TARGET_CLASSIFY	-j CLASSIFY (设置 CBQ 类别)
target	XT_TARGET_CONNMARK	-j CONNMARK (设置连接 mark)
target	XT_TARGET_LOG	-j LOG (记录匹配包到内核日志)
target	XT_TARGET_MARK	-j MARK (设置包 mark 值)
target	XT_TARGET_NFLOG	-j NFLOG (通过 nflog 记录包)
target	XT_TARGET_NFQUEUE	-j NFQUEUE (将包送入用户空间队列)
match	XT_MATCH_ADDRTYPE	-m addrtype (按地址类型匹配)
match	XT_MATCH_COMMENT	-m comment (添加注释)
match	XT_MATCH_CONNMARK	-m connmark (按连接 mark 匹配)
match	XT_MATCH_CONNTRACK	-m conntrack --ctstate (连接跟踪匹配)
match	XT_MATCH_DSCP	-m dscp (按 DSCP 值匹配)
match	XT_MATCH_ESP	-m esp (匹配 ESP/IPSec 包)
match	XT_MATCH_HELPER	-m helper (按连接跟踪 helper 匹配)
match	XT_MATCH_IPRANGE	-m iprange (按 IP 地址范围匹配)
match	XT_MATCH_LENGTH	-m length (按包长度匹配)
match	XT_MATCH_LIMIT	-m limit (按速率限制匹配)
match	XT_MATCH_MAC	-m mac (按 MAC 地址匹配)
match	XT_MATCH_MARK	-m mark (按 mark 值匹配)
match	XT_MATCH_MULTIPORT	-m multiport (多端口匹配)
match	XT_MATCH_OWNER	-m owner (按套接字属主匹配)
match	XT_MATCH_PKTTYPE	-m pkttype (按包类型匹配)
match	XT_MATCH_QUOTA	-m quota (按配额匹配)
match	XT_MATCH_STATE	-m state --state (连接状态匹配, conntrack 别名)
match	XT_MATCH_STATISTIC	-m statistic (统计模式匹配)

类别	选项	支持的 iptables 功能
match	XT_MATCH_STRING	-m string (按包内容字符串匹配)
match	XT_MATCH_TCPMSS	-m tcpmss (按 TCP MSS 值匹配)
match	XT_MATCH_TIME	-m time (按时间匹配)
match	XT_MATCH_U32	-m u32 (通用 u32 匹配)

五、测试命令

烧录新内核后，按以下顺序验证 iptables 扩展功能。

5.1 内核模块验证

```
# 查看内核已注册的 iptables match 列表（修复前只有 5 个，修复后应大幅增加）
cat /proc/net/ip_tables_matches

# 查看内核已注册的 iptables target 列表
cat /proc/net/ip_tables_targets

# 查看 conntrack 连接跟踪表（需要有流量后才有内容）
cat /proc/net/nf_conntrack
```

预期结果：/proc/net/ip_tables_matches 应包含 conntrack、state、comment、limit、mac、multiport、string 等。

5.2 iptables 版本确认

```
iptables --version
# 预期输出: iptables v1.8.7 (legacy)
```

5.3 conntrack 连接跟踪测试

这是最核心的功能验证：

```
# 测试 conntrack 匹配（推荐，v1.8.7 原生接口）
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# 测试 state 匹配（conntrack 的别名接口，v1.8.7 兼容）
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# 查看 conntrack 匹配的帮助信息
iptables -m conntrack --help
```

5.4 基本过滤规则测试

```
# 按 TCP 端口过滤
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# 按 UDP 端口过滤
iptables -A INPUT -p udp --dport 53 -j ACCEPT

# 多端口匹配
iptables -A INPUT -p tcp -m multiport --dports 22,80,443 -j ACCEPT

# 按 MAC 地址过滤
iptables -A INPUT -m mac --mac-source 00:11:22:33:44:55 -j DROP
```

5.5 扩展匹配测试

```
# comment 注释
iptables -A INPUT -p tcp --dport 22 -m comment --comment "Allow SSH" -j ACCEPT

# limit 速率限制（每分钟最多 10 个包）
iptables -A INPUT -p icmp -m limit --limit 10/min -j ACCEPT

# string 内容匹配（匹配 HTTP 请求中的特定字符串）
iptables -A INPUT -p tcp --dport 80 -m string --string "GET /admin" --algo bm -j DROP

# IP 范围匹配
iptables -A INPUT -m iprange --src-range 192.168.1.100-192.168.1.200 -j ACCEPT

# 包长度匹配
iptables -A INPUT -m length --length 64:128 -j ACCEPT

# 按时间匹配（仅工作时间内允许）
iptables -A INPUT -m time --timestart 09:00 --timestop 18:00 --weekdays Mon,Tue,Wed,Thu,Fri -j ACCEPT

# mark 匹配
iptables -A INPUT -m mark --mark 0x1 -j ACCEPT

# connmark 连接标记匹配
iptables -A INPUT -m connmark --mark 0x1 -j ACCEPT

# addrtype 地址类型匹配
iptables -A INPUT -m addrtype --dst-type LOCAL -j ACCEPT

# DSCP 匹配
iptables -A INPUT -m dscp --dscp 46 -j ACCEPT
```

5.6 target 动作测试

```
# LOG 目标（将匹配包记录到 dmesg）
iptables -A INPUT -p tcp --dport 22 -j LOG --log-prefix "SSH access: "

# MARK 设置标记
iptables -A INPUT -p tcp --dport 80 -j MARK --set-mark 0x1

# CONNMARK 设置连接标记
iptables -A INPUT -p tcp --dport 80 -j CONNMARK --set-mark 0x1

# REJECT 拒绝（返回 ICMP 错误）
iptables -A INPUT -p tcp --dport 23 -j REJECT --reject-with tcp-reset

# NFQUEUE 送入用户空间队列（配合用户空间程序处理）
iptables -A INPUT -p tcp --dport 8080 -j NFQUEUE --queue-num 0
```

5.7 NAT 测试

```
# SNAT 源地址转换
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j SNAT --to-source 10.0.0.1

# DNAT 目标地址转换
iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 192.168.1.100:80

# MASQUERADE 动态源地址转换（适合动态 IP 场景）
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE

# REDIRECT 端口重定向
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
```

5.8 规则管理测试

```
# 列出所有规则（带行号）
iptables -L -n -v --line-numbers

# 列出 NAT 表规则
iptables -t nat -L -n -v

# 删除指定规则
iptables -D INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# 按行号删除
iptables -L INPUT --line-numbers
iptables -D INPUT 1

# 清空所有规则
iptables -F

# 保存规则
```



```
iptables-save > /tmp/iptables_rules.txt

# 恢复规则
iptables-restore < /tmp/iptables_rules.txt

# 计数器归零
iptables -Z
```

5.9 典型防火墙场景

```
# 场景 1: 基础防护（允许已建立连接 + 指定端口）
iptables -P INPUT DROP
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# 场景 2: 防端口扫描（限制 SYN 速率）
iptables -A INPUT -p tcp --syn -m limit --limit 1/s --limit-burst 3 -j ACCEPT
iptables -A INPUT -p tcp --syn -j DROP

# 场景 3: IP 黑名单
iptables -A INPUT -s 10.0.0.50 -m comment --comment "Blocked host" -j DROP

# 场景 4: 按包长度过滤异常包
iptables -A INPUT -m length --length 0:40 -j DROP
iptables -A INPUT -m length --length 1501:65535 -j DROP

# 场景 5: 只允许指定 IP 范围访问管理端口
iptables -A INPUT -p tcp --dport 22 -m iprange --src-range 192.168.1.1-192.168.1.10 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j DROP
```

六、注意事项

6.1 iptables 用户空间与内核的对应关系

iptables 的扩展模块采用**双层架构**：

- **用户空间层**（iptables 二进制）：负责解析命令行参数、构建规则结构体
- **内核层**（CONFIG_NETFILTER_XT_MATCH_XXX）：负责实际的数据包匹配操作

两层必须**同时存在**才能正常工作。用户空间编译正确不代表功能可用，还需要内核编译对应模块。

6.2 扩展注册失败的排查方法

如果某个扩展仍然"找不到"，按以下步骤排查：

```
# 1. 检查内核是否支持该 match
cat /proc/net/ip_tables_matches | grep conntrack
```

```
# 2. 检查 iptables 用户空间是否包含该扩展
iptables -m conntrack --help
# 如果输出帮助信息，说明用户空间支持；如果报 "Couldn't find match"，则排查内核

# 3. 检查内核配置
zcat /proc/config.gz 2>/dev/null | grep NETFILTER_XT_MATCH_CONNTRACK
# 或在编译主机检查 defconfig
grep NETFILTER_XT_MATCH_CONNTRACK kernel/linux/config/linux-5.10/arch/arm64/configs/hispanic_hi3519dv500_small_defconfig

# 4. 查看内核日志
dmesg | grep -i conntrack
dmesg | grep -i xtables
```

6.3 版本兼容性

- iptables v1.8.7 使用 conntrack 替代了已移除的 state 模块（-m state 是 -m conntrack 的别名）
- conntrack 支持 revision 0/1/2/3，内核默认支持最高 revision，iptables 用户空间会自动选择内核支持的最高版本
- 如果内核只支持 revision 0（老内核），部分高级参数（如 --ctexpire）可能不可用

6.4 内核体积影响

启用 25 个 NETFILTER 扩展模块会增加内核体积。对于存储空间受限的嵌入式设备，可按需选择启用：

优先级	推荐启用的模块	适用场景
必选	CONNTRACK, STATE, MARK, CONNMARK	基本防火墙、连接跟踪
推荐	COMMENT, LIMIT, MAC, MULTIPORT, LOG	规则管理、日志记录
按需	STRING, TIME, IPRANGE, LENGTH, DSCP, ESP	高级过滤、QoS、IPSec
可选	OWNER, PKTTYPE, QUOTA, STATISTIC, U32, TCPMSS	特定场景

6.5 配置文件位置说明

本项目使用 kernel/linux/config/ 目录管理内核配置：

```
kernel/linux/config/
├─ hisdk/           → 海思 SDK 配置（参考用）
├─ linux-5.10/arch/arm64/configs/
│   └─ hispanic_hi3519dv500_small_defconfig ← 实际构建使用的配置（本文件）
```

修改 kernel/linux/config/linux-5.10/ 下的 defconfig 即可生效。

七、参考资源

1. Linux 内核 netfilter 文档: `kernel/linux/linux-5.10/Documentation/networking/nf_conntrack-sysctl.rst`
2. iptables 扩展源码: `third_party/iptables/extensions/libxt_*.c`
3. iptables 扩展注册逻辑: `third_party/iptables/libxtables/xtables.c` (`xtables_register_match()` 和 `xtables_find_match()`)